

Computer Programming Lab:8

Two-Dimensional Arrays in C++

Prepared by:

Name: Hikmat Ullah Jan

Department: Electrical Comm

Semester: 2nd

Section: B

Reg No: BF25WNELE0662



Objectives

The objective of this lab is to:

- Understand what a two-dimensional array is
- Learn when to use two-dimensional arrays
- Study the declaration and initialization of 2D arrays
- Access and manipulate elements of a two-dimensional array
- Implement programs using 2D arrays

Two-dimensional array

What is an Array? The simplest form of the multidimensional array is the two-dimensional array. A two-dimensional array is, in essence, a list of one-dimensional arrays. To declare a two-dimensional integer array of size x, y, you would write something as follows Syntax: type arrayName [x] [y]; Where type can be any valid C++ data type and array Name will be a valid C++ identifier. A two-dimensional array can be think as a table, which will have x number of rows and y number of columns.

When to Use Two-Dimensional Array?

Use a 2D array when data is organized in **tabular form**, such as:

- Matrices in mathematics
- Student marks (rows = students, columns = subjects)
- Game boards (chess, tic-tac-toe)
- Tables and grids

Declaration:

```
array_name[row][column]
```

Initialization:

Multidimensional arrays may be initialized by specifying bracketed values for each row. Following is an array with 3 rows and each row have 4 columns.

Example:

```
int a[3][4] = { { } {0, 1, 2, 3} ,  
/* initializers for row indexed by 0 */  
{4, 5, 6, 7} ,  
/* initializers for row indexed by 1 */  
{8, 9, 10, 11}  
/* initializers for row indexed by 2 */ ;
```

The nested braces, which indicate the intended row, are optional. The following initialization is equivalent to previous example.

Example:

```
int a[3][4] = {0,1,2,3,4,5,6,7,8,9,10,11};  
  
int matrix[2][3] = {  
    {1, 2, 3},  
    {4, 5, 6}}
```

Accessing Elements:

An element in 2-dimensional array is accessed by using the subscripts, i.e., row index and column index of the array. For example Example: `int val = a[2][3];` The above statement will take 4th element from the 3rd row of the array.

`arr[0][1]` // Access element at first row, second column

Nested Loops:

Used to traverse rows and columns.

Example 1: Input and Display

```

#include <iostream>
using namespace std;
int main() {
    int arr[2][2];
    for(int i = 0; i < 2; i++) {
        for(int j = 0; j < 2; j++) {
            cin >> arr[i][j];
        }
    }
    for(int i = 0; i < 2; i++) {
        for(int j = 0; j < 2; j++) {
            cout << arr[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}

```

Example 2: Sum of Elements

```

#include <iostream>
using namespace std;
int main() {
    int arr[2][2] = {{1,2},{3,4}};
    int sum = 0;

    for(int i = 0; i < 2; i++) {
        for(int j = 0; j < 2; j++) {
            sum += arr[i][j];
        }
    }

    cout << "Sum = " << sum;

    return 0;
}

```

Lab Tasks:

Task 1: 2D Array Declaration, Initialization & Display

```
#include <iostream>           // Include input-output library
using namespace std;         // Use standard namespace

int main() {                  // Main function starts

    int matrix[3][3] = {      // Declare and initialize 3x3 array
        {1, 2, 3},           // First row
        {4, 5, 6},           // Second row
        {7, 8, 9},           // Third row
    };

    cout << "Matrix Elements:\n"; // Display heading

    for(int i = 0; i < 3; i++) { // Loop for rows
        for(int j = 0; j < 3; j++) { // Loop for columns
            cout << matrix[i][j] << " "; // Print each element
        }
        cout << endl;           // Move to next line after each row
    }

    return 0;                 // End of program
}
```

Task 2: Sum of Rows, Columns & Total

```
#include <iostream>
using namespace std;
int main() {
    int arr[3][3] = {
        {2, 4, 6},
        {1, 3, 5},
        {7, 8, 9}
    };
    int totalSum = 0;

    // Calculate row sums
    for(int i = 0; i < 3; i++) {
        int rowSum = 0;
        for(int j = 0; j < 3; j++) {
            rowSum += arr[i][j];
        }
        cout << "Sum of Row " << i+1 << " = " << rowSum << endl; // Display row sum
    }

    // Calculate column sums
    for(int j = 0; j < 3; j++) {
        int colSum = 0;
        for(int i = 0; i < 3; i++) {
            colSum += arr[i][j];
        }
        cout << "Sum of Column " << j+1 << " = " << colSum << endl; // Display column sum
    }

    // Calculate total sum
    for(int i = 0; i < 3; i++) {
        for(int j = 0; j < 3; j++) {
            totalSum += arr[i][j];
        }
    }
    cout << "Total Sum = " << totalSum << endl;

    return 0;
}
```

// Include input-output library
// Use standard namespace
// Main function starts
// Declare and initialize 3x3 array
// Row 1
// Row 2
// Row 3

// Variable to store total sum

// Loop through rows
// Initialize row sum
// Loop through columns
// Add each element of row

// Loop through columns
// Initialize column sum
// Loop through rows
// Add each element of column

// Loop rows
// Loop columns
// Add all elements

// Display total sum

// End program

Task 3: Find Maximum Value in 4x4 Array

```
#include <iostream>
using namespace std;
```

```
int main() {
```

```
    int arr[4][4] = {
        {5, 8, 2, 1},
        {9, 3, 7, 4},
        {6, 10, 12, 11},
        {15, 14, 13, 16}
    };
```

```
    int max = arr[0][0];
```

```
    for(int i = 0; i < 4; i++) {
        for(int j = 0; j < 4; j++) {
            if(arr[i][j] > max) {
                max = arr[i][j];
            }
        }
    }
```

```
    cout << "Maximum value = " << max << endl;
```

```
    return 0;
```

```
}
```

```
// Include input-output library
// Use standard namespace
```

```
// Main function starts
```

```
// Declare and initialize 4x4 array
```

```
// Row 1
```

```
// Row 2
```

```
// Row 3
```

```
// Row 4
```

```
// Assume first element is maximum
```

```
// Loop through rows
```

```
// Loop through columns
```

```
// Check if current element
```

```
// Update maximum value
```

```
// Display maximum value
```

```
// End program
```